NMPC solve time: formulation, root cause, and a path from 100 ms to under 50 ms

carla-track-control · `track_control/src/nmpc.cpp` at commit `24ae556` · 19 October 2026 · all timings measured on this VM (Intel Xeon @ 2.20 GHz, 4 cores / 8 threads)

> **Answer in one paragraph.** The NMPC is slow because **each IPOPT iteration costs about 8 ms**, not because it needs many iterations: a 4 s horizon (N = 40) takes about 13 iterations × 8.1 ms = 105 ms. Of those 8.1 ms, **3.8 ms is evaluating functions in CasADi's interpreted SX virtual machine**. Most of that (2.4 ms) is the exact Hessian of an RK4-integrated Fiala-tire model. The other **4.2 ms is IPOPT's general-sparse KKT solve through MUMPS**, which does not use the stage-by-stage structure of the optimal control problem. Iterations are about 30 % higher than necessary because IPOPT is cold-started on every solve: the previous duals are passed but ignored, and the barrier restarts at μ = 0.1. In the threaded CARLA run, CPU contention raises the per-iteration cost to 12.2 ms. Only 9 iterations then fit in 100 ms, so 98 % of solves hit the wall-time limit. Changing the solver to FATROP (it exploits the OCP structure), compiling the functions, and warm-starting the solver together cut the **mean solve from 105 ms to 19 ms and the 95th percentile from 143 ms to 29 ms**. The lap time stays at 72.3 s, with the same clearance, road margin and sideslip.

105 → 19 ms	**143 → 29 ms**	**51 % → 0 %**	**72.3 s**
mean solve, offline lap (5.6× faster)	95th percentile solve	solves over 100 ms; over 50 ms: 100 % → 0.14 %	lap time before and after; clearance, margin, sideslip unchanged

1. How the NMPC is built

1.1 State, input, parameters

The plan is made in path coordinates. The pose is measured relative to the reference path, so corners enter only through the curvature and the road edges become simple bounds on the lateral offset.

$$x = \begin{bmatrix} s & e_y & e_\psi & U_x & U_y & r & \delta \end{bmatrix}^\top \in \mathbb{R}^7, \qquad u = \begin{bmatrix} \dot{\delta} & a_x = F_x/m \end{bmatrix}^\top \in \mathbb{R}^2, \qquad \sigma \in \mathbb{R}^4_{\geq 0}$$

Here s is arc length, e_y the lateral offset, e_ψ the heading error, U_x, U_y the body-frame velocities, r the yaw rate and δ the road-wheel angle. The longitudinal input is the acceleration rather than the force, which keeps the problem better scaled. Each stage has four slacks σ for the soft constraints (obstacles, road edges, sideslip, speed limit). The parameter vector holds the measured state $\hat{x}$, the curvature κ_k and the speed limit $v_{\text{lim,k}}$ of each step, the predicted arc lengths $s_{j,k}$ of the $n_{\text{cars}} = 2$ nearest cars, and their lateral offsets $e_{j,k}$. κ_k is the mean of five samples of the path curvature over $[s_k, s_{k+1}]$, taken from the previous plan's progress.

1.2 Plant model inside the NMPC (dynamic bicycle, Fiala tires)

Path kinematics:

$$\dot{s} = \frac{U_x \cos e_\psi - U_y \sin e_\psi}{1 - \kappa e_y}, \qquad \dot{e}_y = U_x \sin e_\psi + U_y \cos e_\psi, \qquad \dot{e}_\psi = r - \kappa\, \dot{s}$$

Rigid-body dynamics (a, b are the distances from the centre of gravity to the axles, $L = a + b$):

$$\dot{U}_x = \tfrac{1}{m}\left(F_{xf}\cos\delta - F_{yf}\sin\delta + F_{xr} - F_{\text{drag}}\right) + rU_y, \quad F_{\text{drag}} = \tfrac{1}{2}\rho C_d A\, U_x^2 + C_{rr} mg$$
$$\dot{U}_y = \tfrac{1}{m}\left(F_{yf}\cos\delta + F_{xf}\sin\delta + F_{yr}\right) - rU_x, \qquad \dot{r} = \tfrac{1}{I_{zz}}\left(a(F_{yf}\cos\delta + F_{xf}\sin\delta) - bF_{yr}\right), \quad \dot{\delta} = u_1$$

Drive/brake split and quasi-static longitudinal load transfer (χ is the front share, 0.5 for both drive and brake):

$$F_{xf} = \chi(F_x) F_x, \quad F_{xr} = F_x - F_{xf}, \qquad F_{zf} = \frac{mgb - hF_x}{L}, \quad F_{zr} = \frac{mga + hF_x}{L}$$

Slip angles, with $\tilde{U}_x = \max(U_x, 1\ \text{m/s})$:

$$\alpha_f = \arctan \frac{U_y + ar}{\tilde{U}_x} - \delta, \qquad \alpha_r = \arctan \frac{U_y - br}{\tilde{U}_x}$$

Lateral force from the Fiala brush model, derated by a smoothed friction circle ($F_{\min} = 50$ N keeps the root differentiable when F_x uses all the grip):

$$F_{y,\max} = \sqrt{\tfrac{1}{2}\left(S + \sqrt{S^2 + 4F_{\min}^4}\right)}, \; S = (\mu F_z)^2 - F_x^2, \qquad z = \tan\alpha, \; z_{sl} = \frac{3F_{y,\max}}{C_\alpha}$$

$$F_y = \begin{cases} -C_\alpha z + \dfrac{C_\alpha^2}{3F_{y,\max}}|z|z - \dfrac{C_\alpha^3}{27F_{y,\max}^2} z^3 & |z| < z_{sl} \\ -F_{y,\max}\ \operatorname{sign} z & \text{otherwise} \end{cases}$$

The NMPC holds the cornering stiffness at its static-load value ($C_{\alpha f} = 106\,500$, $C_{\alpha r} = 67\,800$ N/rad). The plant uses the load table, but planning with it doubled the solve time (105 → 209 ms) and did not change the lap. The model is written once as a C++ template and instantiated on `casadi::SX`.

1.3 Discretisation (multiple shooting)

$$x_{k+1} = F_{\text{RK4}}(x_k, u_k; \kappa_k, h), \quad k = 0, \dots, N-1, \qquad x_0 = \hat{x}, \qquad h = 0.1\ \text{s}, \; N = 40\ (4\ \text{s horizon})$$

A single classical RK4 step per interval means four evaluations of the full tire model per stage, or 160 per horizon. The stage variables are interleaved as $[x_0\ u_0\ \sigma_0\ x_1 \dots x_N]$, with each stage's gap-closing constraint first. That is the layout a structure-exploiting solver needs.

1.4 Cost

All state terms are evaluated on the RK4 prediction $x^+_k = F_{\text{RK4}}(x_k, u_k)$, not on the decision variable x_{k+1} (this matters in § 3):

$$J = \sum_{k=0}^{N-1} \Big[w_v (U_x^+ - v_{\max})^2 + w_{\text{lane}} (e_y^+ - e_{\text{ref}})^2 + w_{\text{lat}} \left((v_{\text{lat}}^+)^2 + (r^+ - \kappa_k U_x^+)^2 \right) + w_a a_{x,k}^2 + w_{\dot\delta} \dot\delta_k^2 + w_\sigma \left(\mathbf{1}^\top \sigma_k + \|\sigma_k\|^2 \right) \Big]$$

with $v_{\text{lat}} = U_X \sin e_\psi + U_Y \cos e_\psi$ and weights $w_v = 1$, $w_{\text{lane}} = 0.2$, $w_{\text{lat}} = 1$, $w_a = 1$, $w_{\dot\delta} = 100$, $w_\sigma = 10^3$. No speed profile is prescribed. The cost asks for the speed limit, and the tire friction limit together with the edge and sideslip constraints makes the plan brake for corners.

1.5 Constraints

Constraint (per stage k, on x^+_k)	Expression	rows
Obstacle ellipse, car j	$1 - \left(\frac{s^+ - d_{cg} - s_{j,k}}{A}\right)^2 - \left(\frac{e_y^+ - e_j}{B}\right)^2 \le \sigma_0; B = w_{\text{lane}} - 0.4, A = \frac{l_x}{\sqrt{1-(l_y/B)^2}}, l_x = L_b + 1.0, l_y = W + 0.6$	2
Road edges, front and rear body ends	$\lvert e_y^+ + \ell \sin e_\psi^+ \rvert \le e_{\max} + \sigma_1, \ell \in \{\ell_f, -\ell_r\}, e_{\max} = w_{\text{half}} - W/2 - 0.2$	4
Sideslip	$\lvert U_y^+ \rvert \le \tan(7^\circ)\, U_x^+ + \sigma_2$	2
Speed limit / unseen road	$U_x^+ \le v_{\text{lim},k} + \sigma_3$ (the speed limit where the road is known, beyond it the speed the tightest corner allows)	1
Dynamics (gap closing)	$x_{k+1} - F_{\text{RK4}}(x_k, u_k) = 0$; plus $x_0 - \hat{x} = 0$ once	7 (+7)
Simple bounds	$\lvert \delta_k \rvert \le 0.855, U_{x,k} \ge 3$ m/s ($k \ge 1$); $\lvert \dot\delta_k \rvert \le 0.6$ rad/s; $a_{x,k} \in [F_x^{\min}, F_x^{\max}](U_{x,0})/m; \sigma_k \ge 0$	—

Problem size: 527 variables, 287 equality and 360 inequality constraints, 2087 + 3080 Jacobian nonzeros, 1960 Hessian nonzeros. **Solver:** IPOPT 3.14.11 with MUMPS 5.4.1, exact Hessian, `tol = 1e-6`, `max_iter = 200`, `max_wall_time` = the deadline in fallback mode. The functions are evaluated by CasADi's SX virtual machine (no code generation). The previous primal and dual solutions are passed as `x0`, `lam_x0`, `lam_g0`, with arc length re-based to the new s_0.

Real-time loop (fallback mode). The NMPC solves back to back. Each solve plans from the state predicted one deadline (100 ms) ahead, and its plan is used only if it converged within the deadline. Otherwise pure pursuit's command is sent.

2. Where the 100 ms goes (measurements)

Sources: the CARLA logs in `results/carla/*_solves.csv`, plus 23 offline laps of `build/simulate --controller nmpc --plant bicycle` (default sensing: noise, 60 m range, road known 40 m ahead). Each lap has 1445 solves, and each run was pinned to its own physical core with `OMP_NUM_THREADS=1`. The experiments used env-var switches in a scratch git worktree; the repository was not modified.

Run	mean	p50	p95	max	iter.	ms / iter	evals	Hessian	solver int.	status
CARLA, lockstep NMPC (`nmpc_bicycle`)	107	104	—	161*	13.0	8.2	50	31	57	1447/1447 converged
CARLA, threaded fallback, 100 ms limit	107	107	—	115*	8.8	12.2	44	26	63	905/922 hit `Maximum_WallTime`
Offline baseline (isolated core)	105	101	143	345	13.0	8.1	50	31	55	all converged
Offline baseline, busy hyperthread sibling	164	157	223	554	13.0	12.6	65	39	99	all converged

Times in ms. * 99th percentile. "evals" is the sum of objective, gradient, constraint, Jacobian and Hessian evaluation times; "solver int." is the rest of the solve (KKT factorisation and back-solves, line search, convergence checks, IPOPT bookkeeping).

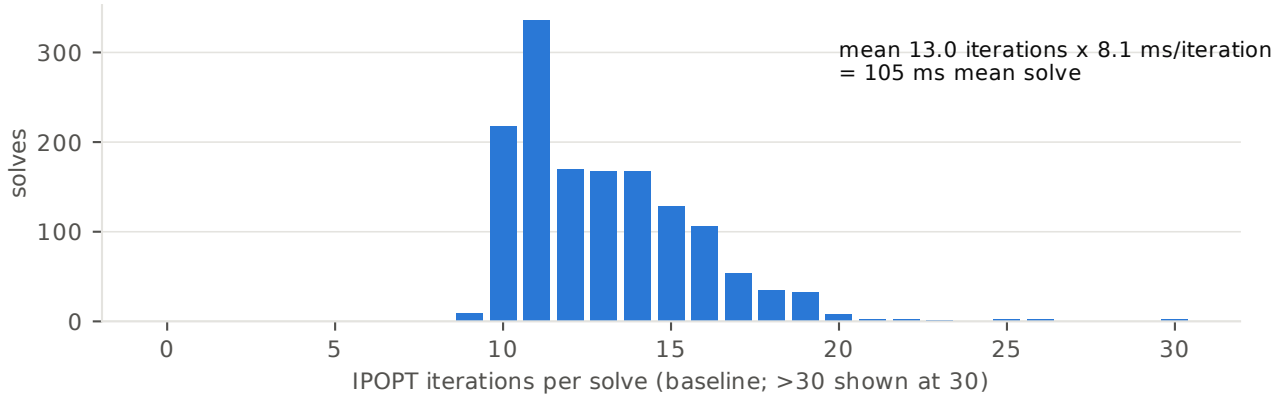


Figure 1. Baseline: 9–20 iterations per solve, 13 on average. The solve time is iterations × per-iteration cost, and the per-iteration cost is the anomaly.

IPOPT's own timing for one typical baseline solve (12 iterations, 126 ms): ComputeSearchDirection 55 ms (MUMPS back-solves 25 ms, the rest IPOPT's augmented-system handling), Lagrangian Hessian 38 ms, inequality Jacobian 12 ms, objective gradient 5 ms, initialisation 7 ms. The iteration log shows the barrier parameter falling from log μ = −1 to −7.0 over the solve. Every solve redoes the whole barrier path, however good the starting point is.

3. Root-cause analysis, ranked

1. **Interpreted function evaluation (≈ 48 % of the time, 3.8 ms/iteration).** The NLP is an SX expression graph run by CasADi's virtual machine, one instruction at a time. The graph is large. Each of the 40 stages has four RK4 stages, each with two Fiala tires (`atan`, `tan`, two nested square roots, and an `if_else` whose branches are both evaluated), plus load transfer. The exact Hessian differentiates all of that twice, and alone takes 2.4 ms per iteration, 31 ms per solve. Because the cost and all nine inequality rows are written on the RK4 output x^+_k instead of on x_{k+1}, the same second derivatives are generated three times (dynamics multipliers, cost, inequalities). Compiling the same graph to C cuts evaluation from 50 ms to 9.8 ms per solve, so most of this cost is interpretation, not arithmetic.
2. **A general sparse KKT solver on a banded problem (≈ 52 %, 4.2 ms/iteration).** Each iteration IPOPT builds and factors a symmetric indefinite system of about 1 500 rows (527 variables, 647 constraints, 360 inequality slacks) with MUMPS. It also runs MUMPS analysis and scaling, inertia checks and vector operations, all through general sparse data structures. The OCP's stage structure (each stage couples only to the next) is not used. FATROP's Riccati recursion over the 40 stages does the same Newton step for 0.63 ms per iteration instead of 4.2 ms. SPRAL, a different general sparse solver, is slower again (126 ms internals).
3. **A cold barrier on every solve (≈ 30 % extra iterations).** `lam_x0` and `lam_g0` are passed, but with `warm_start_init_point = no` (the default) IPOPT discards them and restarts at $\mu_0 = 0.1$. With a proper warm start the mean falls from 13.0 to 8.6 iterations, and the lap is identical.
4. **CPU contention in the threaded CARLA run (×1.5 per iteration).** The plant (1 kHz), pure pursuit, the command thread, the CARLA world thread, the view thread and the Python GIL all share the VM's 4 physical cores. Per-iteration cost rises from 8.1 to 12.2 ms. Offline, a busy loop on the NMPC core's hyperthread sibling gives 12.6 ms, which reproduces it. At 12 ms per iteration only about 8–9 iterations fit in 100 ms, so 98 % of solves stop at `Maximum_WallTime_Exceeded` and pure pursuit drives.
5. **Secondary factors.** Obstacles add about 2 iterations when one is in range (12.0 → 14.1). The warm-start gap $|x_0 - \hat{x}|$ correlates only weakly with iterations ($\rho = 0.16$). In the single-NMPC threaded run, 100 ms-stale inputs pushed iterations from 13 to 31: a feedback loop in which slow solves cause harder solves.

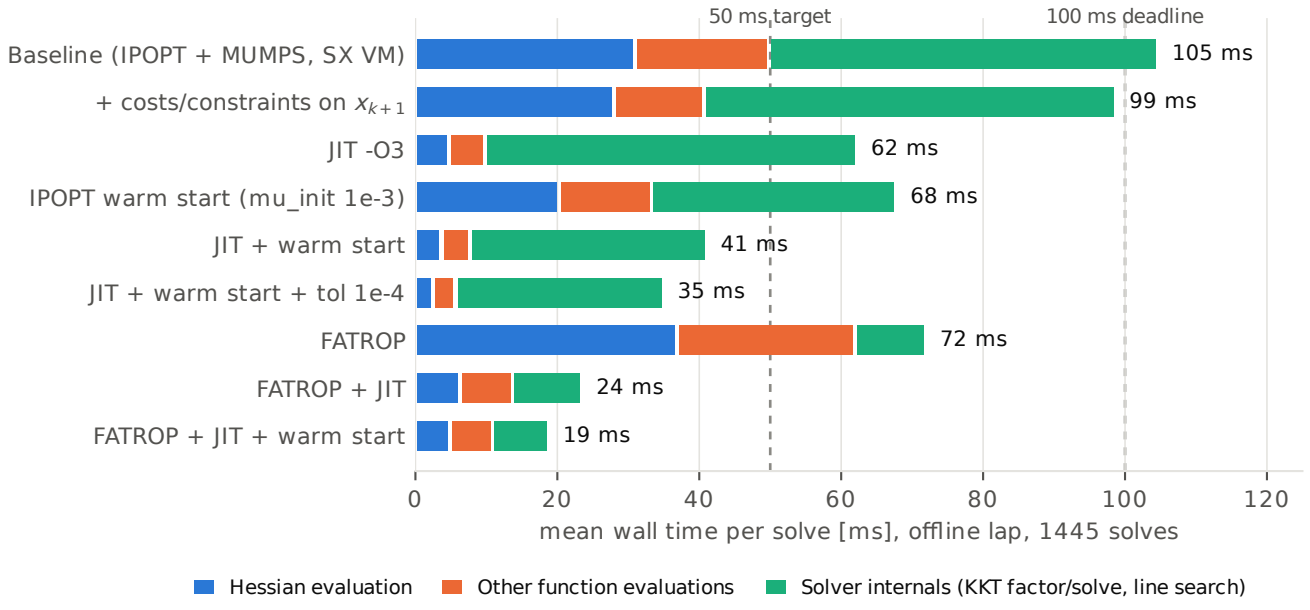


Figure 2. Mean solve time split into Hessian evaluation, other function evaluations, and solver internals. JIT removes most of the blue and orange, FATROP removes most of the green, and warm-starting reduces all three by cutting iterations.

4. Top 10 ways to get under 50 ms without losing performance

"Without loss" was checked, not assumed. Every measured row below drove the full offline lap and was compared with the baseline on lap time (72.3 s), minimum clearance to cars (1.04 m), asphalt margin (0.22 m), peak sideslip (7.0°), minimum speed (41.7 km/h) and convergence (0 failures). measured marks a result from a lap on this machine; estimate marks one not measured.

Configuration	mean	p95	max	>50 ms	iter.	lap [s]	clear. [m]	margin [m]	β_{max} [°]
Baseline	104.7	142.9	345	100 %	13.0	72.3	1.04	0.22	6.97
Costs/constraints on x_{k+1} (#6)	98.7	132.9	294	100 %	13.0	72.3	1.04	0.22	6.97
IPOPT warm start, $\mu_0 = 10^{-3}$ (#3)	67.8	125.6	329	59 %	8.6	72.3	0.96	0.22	7.00
JIT -O3 (#2)	62.2	83.8	227	86 %	13.0	72.3	1.04	0.22	6.97
FATROP (#1)	71.9	92.4	214	100 %	15.7	72.2	0.99	0.19	7.03
JIT + IPOPT warm start, $\mu_0 = 10^{-2}$	45.9	73.3	241	32 %	9.7	72.3	1.00	0.22	6.99
… + time-shifted warm start (#7)	44.5	70.1	171	27 %	9.5	72.3	0.96	0.22	6.98
JIT + IPOPT warm start, $\mu_0 = 10^{-3}$	41.0	75.6	214	25 %	8.6	72.3	0.96	0.22	7.00
… + x_{k+1} costs + time shift	38.8	68.8	306	18 %	8.4	72.3	0.96	0.22	7.00
JIT + warm start + tol 10^{-4} (#4), best IPOPT	35.0	68.4	207	15 %	7.4	72.3	0.96	0.22	6.99
FATROP + JIT	23.5	29.9	72	0.14 %	15.7	72.2	0.99	0.19	7.03
FATROP + JIT + FATROP warm start (recommended)	**18.8**	**29.1**	**95**	**0.14 %**	**13.0**	**72.3**	**0.96**	**0.22**	**7.00**
FATROP + JIT, busy hyperthread sibling (#5)	36.4	46.4	114	1.5 %	15.7	72.2	0.99	0.19	7.03
Horizon 3 s (rejected)	58.6	154.9	916	34 %	14.9	left track at 25.4 s	1.09	−3.32	9.74
IPOPT warm start, $\mu_0 = 10^{-4}$, pushes 10^{-6} (rejected)	71.4	155.9	774	57 %	8.9	74.8	1.01	0.21	7.11
JIT + SPRAL instead of MUMPS (rejected)	140.3	194.4	624	100 %	13.0	72.3	1.04	0.22	6.97

Solve times in ms. Clearance is the minimum distance to another car; margin is the minimum distance from the body to the asphalt edge; β_{max} is the peak sideslip. FATROP converges to the same optimum along a different path, so the lap differs from IPOPT's by 0.1 s and a few centimetres, within run-to-run spread.

Robustness across sensor-noise seeds (baseline vs the recommended FATROP + JIT + warm start):

Seed	Setup	mean	p95	max	>50 ms	lap [s]	clear. [m]	margin [m]	β_{max} [°]	min speed [km/h]
1 (default)	baseline	104.7	142.9	345	100 %	72.3	1.04	0.22	6.97	41.7
	recommended	**18.8**	**29.1**	**95**	**0.14 %**	**72.3**	**0.96**	**0.22**	**7.00**	**41.7**
2	baseline	104.8	145.3	292	100 %	72.2	0.90	0.20	7.06	42.6
	recommended	**19.7**	**31.0**	**115**	**0.76 %**	**72.3**	**0.95**	**0.22**	**7.05**	**42.8**
3	baseline	105.6	150.6	484	100 %	72.2	0.93	0.20	7.08	42.0
	recommended	**19.4**	**29.5**	**78**	**0.62 %**	**72.5**	**1.02**	**0.22**	**7.11**	**42.5**

Over the three seeds the lap times differ by 0.0, +0.1 and +0.3 s, and clearance, asphalt margin and sideslip stay within the baseline's own spread (sideslip at most 7.11° against the 7° soft limit). None of the 4 335 solves failed. The worst single solve is 115 ms, so the deadline logic and pure-pursuit fallback should stay in place.

1. **Switch the NLP solver to FATROP** measured: internals 55 → 10 ms; 105 → 72 ms alone, 24 ms with JIT
   FATROP is an interior-point solver that factors the KKT system by a Riccati recursion over the stages: O(N) dense block operations on 7×7/13×13 blocks through BLASFEO, instead of a general sparse LDL^T. It ships with the CasADi 3.7.2 wheel already in `.venv`. The NLP is already written stage-wise for it, and the existing `TC_NMPC_OPTS` hook passes the `equality` flags. One fix is needed in `nmpc.cpp`: FATROP's `return_status` is not a string, so `st.at("return_status").to_string()` throws after the first solve; read it only if `is_string()`. Options: `structure_detection=auto, fatrop.tol=1e-6, fatrop.max_iter=200, fatrop.print_level=0`. Effect on the lap: 72.2 s, clearance 0.99 m, margin 0.19 m, 0 failures.

2. **Compile the NLP functions instead of interpreting them** measured: evaluations 50 → 9.8 ms; Hessian 31 → 4.8 ms
   Set `jit = true, compiler = "shell", jit_options.flags = "-O3"` on `nlpsol`. It gives the same numbers, so the lap is unchanged to the millimetre. **Caveat (measured):** JIT-compiling the Hessian at `-O3` takes about 7–8 minutes at start-up on this VM (lap wall time 531 s against 105 s without JIT). Do not compile at run time in CARLA. Generate the C code once at build time (`solver.generate_dependencies("track_nmpc.c")`, or `casadi::external` on a prebuilt `.so`) and cache it by a hash of the options, track and vehicle parameters. Compiling at `-O1` is not a way around this: with the recommended setup it gave a 22 ms mean (against 19 ms at `-O3`) and still took about 5.5 minutes to compile (measured).

3. **Warm-start the barrier as well as the primal** measured: 13.0 → 8.6 iterations; 62 → 41 ms with JIT
   IPOPT: `warm_start_init_point=yes, mu_init=1e-3, warm_start_bound_push=1e-4, warm_start_mult_bound_push=1e-4`. FATROP: `fatrop.warm_start_init_point=true, fatrop.mu_init=1e-2` (23.5 → 18.8 ms). The size of μ_0 matters. $\mu_0 = 10^{-4}$ with pushes of 10^{-6} slowed the lap to 74.8 s (minimum speed 41.7 → 32.6 km/h) and produced a 774 ms solve, because the iterate sticks to active bounds. Keep μ_0 between 10^{-3} and 10^{-2}, and keep a cold-start fallback after a failed solve.

4. **Use real-time tolerances** measured: 41 → 35 ms, 8.6 → 7.4 iterations, same lap
   `tol=1e-4, acceptable_tol=1e-3, acceptable_iter=2`. A plan whose first input is held for 10–100 ms does not need a 10^{-6} KKT residual. The last two iterations change the inputs by far less than the actuator lags do. With FATROP the equivalent is `fatrop.tol=1e-4` (estimate: another 1–2 iterations saved).

5. **Give the NMPC thread an uncontended physical core** measured: contention costs ×1.55 per iteration (8.1 → 12.6 ms)
   In the CARLA run, contention alone is the difference between 13 iterations fitting in 100 ms and 9 fitting. Pin the `nmpc` thread with `pthread_setaffinity_np` to a core whose hyperthread sibling is left idle, and keep the plant, pure pursuit, the world and the view threads on the other cores. Set `OMP_NUM_THREADS=1` so MUMPS/OpenBLAS do not oversubscribe, and use `SCHED_FIFO` or a lower nice value where allowed (the plant thread already tries this). With FATROP + JIT the contended mean is 36 ms and the p95 46 ms, still under 50 ms.

6. **Put the cost and path constraints on x_{k+1}, not on the RK4 output** measured: 105 → 99 ms; gradient 4.1 → 0.3 ms
   At convergence $x_{k+1} = F_{\mathrm{RK4}}(x_k, u_k)$, so the optimum is identical (the lap matches the baseline exactly). The cost becomes a simple quadratic in the variables, and the inequality Jacobian becomes nearly constant. This removes two copies of the RK4 second derivatives from the Hessian. The gain is small after JIT. For FATROP, write the path constraints as functions of stage k's own x_k (shifting the index by one, with the terminal constraints on x_N). Placing them right after the gap-closing row of stage k breaks FATROP's structure detection (measured).

7. **Shift the warm start in time** measured: max 241 → 171 ms, mean 45.9 → 44.5 ms
   The warm start currently re-bases only s. Shifting the whole primal trajectory by the elapsed time (one step of $h = 0.1$ s in fallback mode, half a step in lockstep), then interpolating and repeating the last stage, gives the solver a guess that already matches where the car now is. Most of the benefit is in the tail, which is what the deadline depends on.

8. **Make the tire algebra cheaper while keeping it exact** estimate: 10–25 % fewer Hessian operations
   The model computes $\alpha = \operatorname{atan}(\cdot)$ and then $z = \tan\alpha$. For the rear axle $\tan\alpha_r = (U_y - b\,r)/\tilde{U}_x$ exactly; for the front, $\tan(\theta - \delta) = (t - \tan\delta)/(1 + t\tan\delta)$. That removes four transcendental functions and their second derivatives from each RK4 stage. Sharing $1/F_{y,\max}$ and $C_\alpha/F_{y,\max}$ between the three polynomial terms, and computing F_z and the drag once per stage, also helps. The results are identical, so performance is unaffected.

9. **Drop the x_0 equality and fixed bounds from the KKT system** estimate: 2–5 % per iteration
   Replace $x_0 - \hat{x} = 0$ by fixing x_0 through `lbx = ubx = x̂` (IPOPT's default `fixed_variable_treatment=make_parameter` then removes them), or substitute $\hat{x}$ directly. This removes 7 variables and 7 equality rows. In the same spirit, the 80 input bounds and 160 slack bounds are already simple bounds, which is right. Keep them out of `g`.

10. **Once under 50 ms, halve the latency compensation** estimate: fewer iterations and fresher inputs
    Fallback mode plans from the state predicted one full deadline ahead. With a 50 ms deadline (`--nmpc-deadline 0.05`) the prediction spans half the time, so the warm start is closer and the applied plan is 50 ms fresher. That is a performance *gain*, and it removes the slow-solve → stale-input → harder-solve loop seen in the threaded single-NMPC run (13 → 31 iterations). If even more margin is ever needed, the next step is real-time iteration (one SQP step per sample, with HPIPM as the QP solver, also in the wheel). It changes the optimality semantics, though, so it needs its own lap validation.

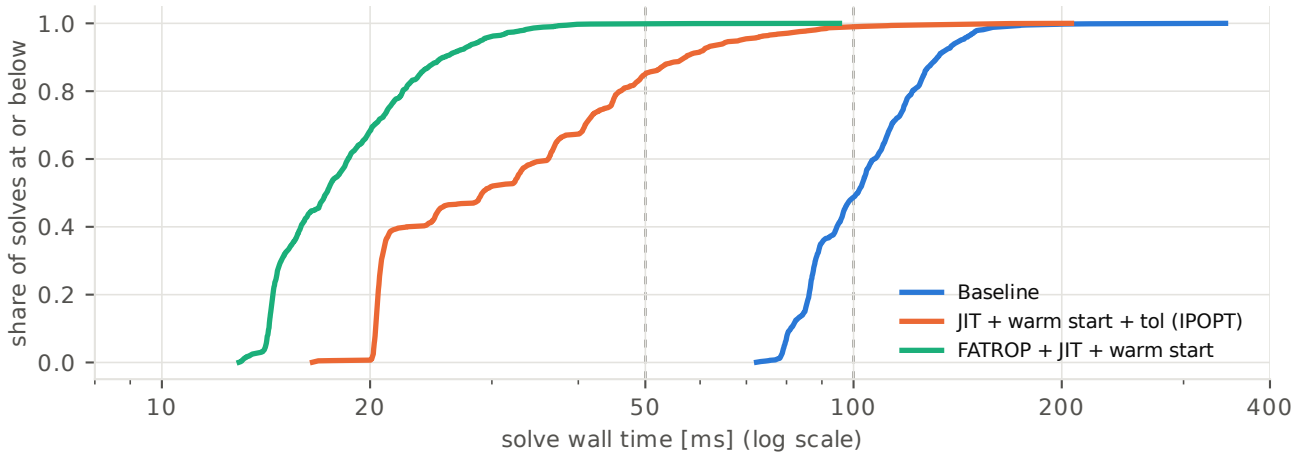


Figure 3. Solve-time distribution. Baseline: half the solves are over 100 ms. Best IPOPT setup (JIT + warm start + tol): mean 35 ms, but a long tail with 15 % over 50 ms. FATROP + JIT + warm start: 99.86 % under 50 ms, p95 29 ms.

5. What not to do

- **Shortening the horizon**, the earlier report's open item 1. With a 3 s horizon (N = 30) the car **left the track at 25.4 s**, 3.3 m past the asphalt edge, with a sideslip of 9.7°. With the road known only 40 m ahead and corners braked for through the model, the horizon is what gives the plan time to brake. It is a performance parameter, not a tuning knob for solve time.
- **A different general sparse solver.** SPRAL is 2.3× slower than MUMPS here. HSL MA27/MA57 are not in the wheel; they would help IPOPT somewhat, but would not reach FATROP's structure-exploiting cost.

- **An over-aggressive warm start** ($\mu_0 \leq 10^{-4}$, tiny bound pushes): measurably slower lap, and the worst tail of all runs.
- **A coarser step or RK2.** Not measured here. It changes the prediction the constraints are enforced on, so it trades accuracy for time and would need its own validation.

6. Recommended change set and how to verify it

1. In `nmpc.cpp`, make the solver an `Options` field (`"ipopt"` / `"fatrop"`) instead of the `TC_NMPC_OPTS` experiment. Fix the `return_status` read. Pass the FATROP warm-start options when `m.warm` is true.
2. Generate and compile the solver's functions ahead of time, from CMake or on first run, and cache the result. Load it with `casadi::external`.
3. Pin the `nmpc` thread in `FallbackControl::nmpc_loop` next to the existing `pthread_setname_np`, and give the plant its own core.
4. Then optionally apply #6–#9, and lower `--nmpc-deadline` to 0.05.
5. Verify: `ctest`, then `pytest` (the full laps), offline laps on three seeds against the baseline numbers above, then the CARLA fallback run. The target there is an `nmpc_command_share` near 1 (it is about 1/140 today).

Appendix: reproducing the measurements

```
# scratch worktree build with env switches (not committed): EXP_JIT=<flags>, EXP_STAGE=1, EXP_SHIFT=1,
EXP_HORIZON=<s>
OMP_NUM_THREADS=1 taskset -c 0 build/simulate --controller nmpc --plant bicycle --out runs/base
OMP_NUM_THREADS=1 EXP_JIT=-O3 TC_NMPC_OPTS=plugin=fatrop,structure_detection=auto,print_time=false,\
  fatrop.print_level=0,fatrop.tol=1e-
6,fatrop.max_iter=200,fatrop.warm_start_init_point=true,fatrop.mu_init=1e-2 \
  taskset -c 0 build/simulate --controller nmpc --plant bicycle --out runs/best
# IPOPT timing breakdown of every solve:
TC_NMPC_OPTS=ipopt.output_file=ipopt.out,ipopt.file_print_level=5,ipopt.print_timing_statistics=yes
build/simulate ...
```